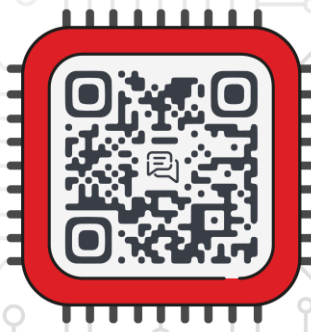




عملي

1

19

2280 sp

كلية الهندسة المعلوماتية

السنة الثانية

مراجعة أساسيات البرمجة غرضية التوجه OOP

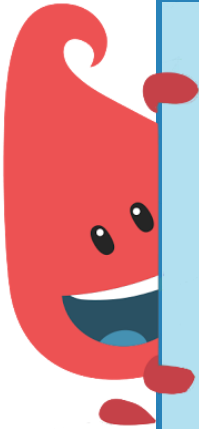
عملي مشترك



RB Informatics; 25/10/2024

برمجة ٣

بسم الله الرحمن الرحيم



زملاءنا ورفاقنا، بعد أن تعرفنا في مادة البرمجة 2 على مبادئ البرمجة غرضية التوجه، سندرس في البرمجة 3 أبحاثاً إضافية، وستتركز محاضرات العملي على حل تمارين وتطبيق أمثلة على أفكار محاضرات النظري.

سنراجع في المحاضرات الأولى ما درسناه في البرمجة 2 مع تطبيق أمثلة عليها ودراسة بعض الملاحظات.

نأمل لكم ولنا فضلاً دراسياً موفقاً، لا تنسونا من صالح دعائكم.

The only limit to your impact is your imagination and commitment!

التمرين الأول:

عرف Class باسم Student يحوي :

■ ال attributes التالية :

1- Name (الاسم)

2- Marks array (مصفوفة العلامات)

■ أيضاً methods و هي :

1- Default constructor (باني افتراضي)

2- Parameterized constructor (باني مع قيم ممررة)

3- Average calculation method (لحساب معدل الطالب)

4- Print method (لطباعة معلومات الطالب)

ثم ضمن التابع main نقوم بعمل Object من Student Class من خلال جعل المستخدم يمرر القيم و ثم طباعة معلومات هذا ال Object.

File: Student.java

```
1. Package oop_concept;
2. //تضمين المكتبة للاستفادة من توابعها
3. import java.util.Scanner;
4. //Student class
5. public class Student {
6.     /*the attributes*/
7.     String name;
8.     int [] marks = new int[3];
9.
10.    /*default constructor*/
11.    public Student(){
12.    }
13.
14.    /*parameterized constructor*/
15.    public Student(String name, int marks[] ){
16.        //إسناد قيم للاسم و لمصفوفة العلامات
17.        this.name = name;
18.        for(int i=0; i<marks.length; i++)
19.            this.marks[i]=marks[i];
20.    }
21.
22.    /*average calculation method*/
23.    double avg(){
24.        //تعريف متغير لحفظ مجموع العلامات
25.        double sum=0.0;
26.
27.        //تجميع العلامات من خلال المرور على قيم مصفوفة العلامات كلها
28.        for(int i=0; i<marks.length; i++)
29.            sum +=marks[i];
30.
31.        //نقسم مجموع العلامات على عددها لنحصل على المتوسط الحسابي
32.        return sum/3;
33.    }
34.
35.    /*print information method*/
36.    void printInfo(){
37.        // طباعة الاسم
38.        System.out.println("name: " +name);
39.        // طباعة العلامات
40.        for(int i=0; i<marks.length; i++)
41.            System.out.println(marks[i]);
42.        // طباعة المتوسط الحسابي للعلامات
43.        System.out.println("average: " + avg());
44.    }
```

```

45. public static void main(String[] args) {
46.     // تعريف أوبجكت من "scanner" لأخذ البيانات المطلوبة من المستخدم
47.     Scanner input = new Scanner(System.in);
48.     // تعريف متغير الاسم و مصفوفة العلامات لإرسالها لباني القيم
49.     String name;
50.     int [] marks = new int[3];
51.
52.     // إدخال الاسم
53.     System.out.println("Enter student name: ");
54.     name= input.next();
55.
56.     // إدخال العلامات
57.     for (int i=0; i<marks.length; i++){
58.         System.out.println("Enter Mark " + i);
59.         marks[i]= input.nextInt();
60.     }
61.
62.     // إنشاء كائن من "Student" مستخدمين باني القيم من خلال إرسال الاسم و مصفوفة العلامات
63.     Student student = new Student(name,marks);
64.     System.out.println();
65.     System.out.println("Student Info: ");
66.     // استدعاء method الطباعة للمعلومات
67.     student.printInfo();
68. }

```

طلب إضافي أول على التمرين الأول:

قم بفصل student class عن ال main class

الحل:

File: Student.java

```

1. Package oop_concept;
2. //Student Class
3. public class Student {
4.     /*the attributes*/
5.     String name;
6.     int [] marks = new int[3];
7.
8.     /*default constructor*/
9.     public Student(){
10.    }

```

File: Main.java

```

1. Package oop_concept;
2. //تضمين المكتبة للاستفادة من توابعها
3. import java.util.Scanner;
4. //Main Class
5. class Main {
6.     public static void main(String[] args) {
7.         // تعريف أوبجكت من "scanner" لأخذ البيانات
            المطلوبة من المستخدم
8.         Scanner input = new Scanner(System.in);
9.         // تعريف متغير الاسم و مصفوفة العلامات
            لإرسالها لباني القيم
10.        String name;

```

```

11.  /*parameterized constructor*/
12.  public Student(String name, int marks[] ){
13.      this.name = name;
14.      for(int i=0; i<marks.length; i++)
15.          this.marks[i]=marks[i];
16.  }
17.
18.  /*average calculation method*/
19.  double avg(){
20.      //making a double to store the sum
21.      double sum=0.0;
22.      // get the sum of the marks
23.      for(int i=0; i<marks.length; i++)
24.          sum +=marks[i];
25.      //divide the marks sum by the number of marks
26.      return sum/3;
27.  }
28.
29.  /*print information method*/
30.  void printInfo(){
31.      // printing the name
32.      System.out.println("name: " +name);
33.      //printing the marks
34.      for(int i=0; i<marks.length; i++)
35.          System.out.println(marks[i]);
36.      //printing the average
37.      System.out.println("average: " + avg());
38.  }
39. }

```

```

11.  int [] marks = new int[3];
12.  // إدخال الاسم
13.  System.out.println("Enter student
name: ");
14.  name= input.next();
15.
16.  // إدخال العلامات
17.  for (int i=0; i<marks.length; i++){
18.      System.out.println("Enter Mark "
+ i);
19.      marks[i]= input.nextInt();
20.  }
21.
22.  // إنشاء كائن من "Student" مستخدمين
باني القيم من خلال إرسال الاسم ومصفوفة
العلامات
23.  Student student = new
Student(name,marks);
24.  System.out.println();
25.  System.out.println("Student Info: "
);
26.  // استدعاء method للطباعة للمعلومات
27.  student.printInfo();
28. } }

```

طلب إضافي ثانٍ على التمرين الأول:

سنعيد كتابة محتويات Main class

- نقوم بكتابة تابعين:
 - maxAvg تابع يقوم بإيجاد قيمة أعلى معدل بين معدلات مجموعة من الطلاب.
 - maxAvgStudent تابع يقوم بإيجاد صاحب أعلى معدل بين مجموعة من الطلاب.
- ثم نقوم بتعريف التابع main وضمنه نعرّف مصفوفة من الطلاب و ندع المستخدم يدخل بيانات الطلاب.
- ثم نطبع:
 - معلومات جميع الطلاب.
 - أعلى معدل بين جميع الطلاب.
 - وبيانات الطالب صاحب أعلى معدل.
- مستفيدين من التوابع السابقة.

File: Main.java

```
1. Package oop_concept;
2. import java.util.Scanner;
3. //Main Class
4. public class Main {
5. public static void main(String[] args) {
6.     // Scanner object
7.     Scanner input = new Scanner(System.in);
8.     // متغير لتخزين الاسم
9.     String name;
10.    // مصفوفة العلامات
11.    int [] marks = new int[3];
12.    // مصفوفة الطلاب المطلوبة
13.    Student [] students = new Student[3];
14.    // حلقة لإدخال الطلاب طالباً طالباً
15.    for (int j=0; j<students.length;j++){
16.        System.out.println( );
17.        System.out.println("Enter student " +j+ " Info: " );
18.        System.out.println("Enter student name: " );
19.        // تخزين اسم الطالب الحالي
20.        name= input.next();
21.        // حلقة لإدخال علامات الطالب الحالي
22.        for (int i=0; i<marks.length; i++){
23.            System.out.println("Enter Mark " + i);
24.            marks[i]= input.nextInt();
25.        }
26.        // إضافة الطالب الحالي إلى المصفوفة الخاصة بالطلاب
27.        students[j] = new Student(name,marks);
28.    }
29.    // حلقة لطباعة بيانات كافة الطلاب
30.    for (int j=0; j<students.length;j++)
31.    {
32.        System.out.println( );
33.        System.out.println("Student " +j+ " Info: " );
34.        students[j].printlnInfo();
35.    }
36.    // طباعة أعلى معدل
37.    System.out.println("The Max Avg is " + MaxAvg(students) );
38.    System.out.println( );
39.    // طباعة بيانات الطالب صاحب أعلى معدل
40.    System.out.println("The Max Avg Student Info is :");
41.    // ملاحظة : يقوم التابع التالي بإرجاع أوبجكت من نوع طالب يمكنها الوصول للتابع
printlnInfo */
42.    maxAvgStudent(students).printlnInfo();
43. }
```

```

44. static double MaxAvg (Student students[]){
45.     // متغير لتتبع القيمة العظمى للمعدل نعطيها قيمة ابتدائية قيمة معدل أول طالب
    بمصفوفة الطلاب
46.     double max=students[0].avg();
47.     // حلقة للمرور على كافة طلاب المصفوفة إلا أول طالب
48.     for (int j=1; j<students.length;j++){
49.         // إذا كان أعلى معدل لدي الآن أصغر من معدل الطالب الحالي فاجعل معدل الطالب
        الحالي هو أكبر معدل بدلاً من القيمة السابقة
50.         if (max<students[j].avg())
51.             max=students[j].avg();
52.     }
53.     // إرجاع أعلى معدل
54.     return max;
55. }
56. static Student maxAvgStudent (Student students[]){
57.     // متغير لتتبع المعدل الأعظمي كما سبق نعطيها قيمة معدل أول طالب لدينا
58.     double max=students[0].avg();
59.     // متغير من نوع طالب لنحفظ الطالب ذا المعدل الأعلى عند الانتهاء من البحث
60.     Student student = students[0];
61.     for (int j=1; j<students.length;j++){
62.         // إذا كان معدل الطالب أعلى من المعدل الأعظمي الذي أمتلكه الآن اجعل المعدل
        الأعظمي هو معدل هذا الطالب و اجعل متغير student هو هذا الطالب من المصفوفة
63.         if (max<students[j].avg()){
64.             max=students[j].avg();
65.             student=students[j];
66.         }
67.     }
68.     // إرجاع الطالب ذي المعدل الأعظمي
69.     return student;
70. }
71. }

```

التمرين الثاني:

عرف Class باسم Time فيه:

■ Attributes

hours لتمثيل الساعات.

minutes لتمثيل الدقائق.

seconds لتمثيل الثواني.

■ Methods

Parameterized constructor (باني القيم)

Difference calculator (تابع لحساب فرق التوقيت)

ثم نقوم بتعريف التابع main و نقوم بعمل وقتين من اختيارنا و طباعتهما ثم طباعة فرق الوقت بينهما مستفيدين من التابع السابق.

الحل:

File: Time.java

```
1. Package oop_concept;
2. //Time Class
3. public class Time {
4.     /*The Attributes*/
5.     int seconds;
6.     int minutes;
7.     int hours;
8.     /*Parameterized Constructor*/
9.     public Time(int hours, int minutes, int
seconds) {
10.         this.hours = hours;
11.         this.minutes = minutes;
12.         this.seconds = seconds;
13.     }
14.     /*time difference method*/
15.     // التابع يتلقى توقيتين
16.     public static Time difference(Time start,
Time stop)
17.     {
18.         // نقوم بعمل أوبجكت من الوقت لحفظ
الفرق بين التوقيتين
19.         Time diff = new Time(0, 0, 0);
20.         // نقوم بالتأكد هل الوقت الأول قبل الوقت الثاني
21.
22.         if (start.hours < stop.hours ||
23.             (start.hours == stop.hours &&
start.minutes < stop.minutes) ||
24.             (start.hours == stop.hours &&
start.minutes == stop.minutes && start.seconds
< stop.seconds)) {
25.             // في حال تحقق الشرط نبدل بين الوقت
الأول و الثاني
26.             Time temp = start;
27.             start = stop;
28.             stop = temp;
29.         }
30.         // هدف هذا هو توحيد العمليات في الخطوة
التالية باعتبار الوقت الأول دائماً سيكون أكبر من الوقت
الثاني
```

ملاحظة أريسيزية (شرح التابع printf()):

يقوم التابع بطباعة string بتنسيق خاص يكتب على الشكل الآتي:

Printf("format",value1,value2,value3.....);

حيث ال format يعبر عن ال string الذي سيتم طباعته وفيه يمكن أن نستخدم placeholders التي هي رموز نضعها في مكان ما في format تشير إلى أن الأماكن التي وضعنا فيها الرموز تتوقع أن يكون هنالك متغيرات من نوع ما (يجب تحديد نوعها) ستحل محل الرموز في هذه الأماكن.

وال values هي القيم التي ستشغل هذه الأماكن بالترتيب (أحياناً تكون غير ضرورية في حالة عدم وجود placeholders).

طبعا لاستخدام ال placeholders نضع الرمز في المكان الذي نريده أن يستقبل متغيراً ما نحدده لاحقاً.

هذه بعض من ال placeholders لأنواع البيانات الممكنة البسيطة:

%d : رمز يتلقى عدداً صحيحاً int

%s : رمز يتلقى نصاً string

%f : رمز يتلقى عدداً عشرياً float

%c : رمز يتلقى حرفاً char

أمثلة:

من التمرين السابق:

System.out.printf(" = %d:%d:%d\n", diff.hours, diff.minutes, diff.seconds);

سيكون ال String من الشكل: int:int:int

كل int سيأخذ قيمة من البارمترات من اليسار إلى اليمين بالترتيب

=hours:minutes:seconds

مثال آخر:

System.out.printf("Name: %s, Age: %d, City: %s\n", "Alice", 30, "New York");

سنضع الشكل النهائي مباشرة:

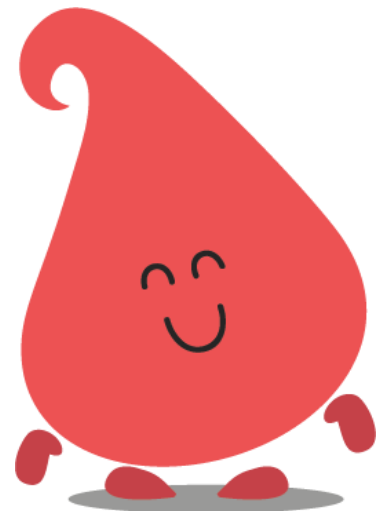
Name: Alice, Age: 30, City: New York

تتمة الملاحظة الأريسيزية (تدرس بعد المحاضرة الثانية نظري):

لو وضعنا placeholders عددها أكبر من القيم التي مرزناها سنحصل على unchecked exception، أما لو مرزنا قيمةً عددها أكبر من ال placeholders فلن يحدث أي شيء سوى تجاهل القيم الأخيرة التي ضفناها في النهاية.

أيضاً لو وضعنا placeholder لنوع بيانات معين ثم مرزنا قيمة من نوع آخر فهذا سيؤدي إلى unchecked exception.

I think somehow we learn who we really are and then live with that decision.



```

31. // إذا كانت الثواني في الوقت الأكبر أقل من
    الوقت الأصغر
32. if(stop.seconds > start.seconds){
33. // طرح دقيقة من الوقت الأكبر و نزيد لثوانيه 60
    ثانية (عملية استلاف بسيطة حيث حولنا دقيقة إلى ستين
    ثانية من أجل عملية الطرح)
34. --start.minutes;
35. start.seconds += 60;
36. }
37. // طرح ثواني العدد الأصغر من ثواني العدد
    الأكبر و نقوم بإسنادها إلى ثواني الفرق
38. diff.seconds = start.seconds -
stop.seconds;
39.
40. // إذا كانت الدقائق في الوقت الأكبر أقل من
    الوقت الأصغر
41. if(stop.minutes > start.minutes){
42. // طرح ساعة من الوقت الأكبر و نزيد لدقائقه
    60 دقيقة
43. --start.hours;
44. start.minutes += 60;
45. }
46. // طرح دقائق الوقت الأصغر من دقائق الوقت
    الأكبر و نسندها لدقائق الفرق
47. diff.minutes = start.minutes -
stop.minutes;
48. // طرح ساعات الوقت الأصغر من ساعات الوقت
    الأكبر و نسندها لدقائق الفرق
49. diff.hours = start.hours - stop.hours;
50.
51. // نرجع متغير الفرق من نوع Time
52. return(diff);
53. }
54. public static void main(String[] args) {
55. // كائن من نوع Time يمثلان وقتين عشوائيين
56. Time start = new Time(12, 34, 55),
57. stop = new Time(8, 12, 15);
58. // كائن من نوع Time سيأخذ القيمة المرجعة من
    فرق هذين الوقتين باستخدام الدالة السابقة
59. Time diff= difference(start, stop);
60. // طباعة الفرق بينهما
61. System.out.printf("= %d:%d:%d\n",
diff.hours, diff.minutes, diff.seconds);
62. }
63. }

```

طلب إضافي على التمرين الثاني:

- اكتب كلاس Time بشكل منفصل عن كلاس Main.
- قم بإزالة تابع حساب فرق الوقت من كلاس Time و قم بعمل override لتابع toString لطباعة التوقيت بشكل مباشر.
- وفي كلاس Main عرف تابعاً يقوم بالتأكد هل الوقت المدخل صحيح أم لا.
- و ثم في التابع main قم بجعل المستخدم يدخل توقيتاً و أخبره إن كان الوقت الذي أدخله صحيحاً أم لا، و إن كان صحيحاً أنشئ object منه ثم اطبعه.

الحل:

File: Main.java

```

1. Package oop_concept;
2. import java.util.Scanner;
3. //Main Class
4. public class Main {
5.     public static void main(String []args)
6.     {
7.         Scanner scanner = new Scanner (System.in);
8.         System.out.print("Enter hours (0-23): ");
9.         int hours = scanner.nextInt();
10.
11.         System.out.print("Enter minutes (0-59): ");
12.         int minutes = scanner.nextInt();
13.
14.         System.out.print("Enter seconds (0-59): ");
15.         int seconds = scanner.nextInt();
16.
17.         // مرور الأرقام للتابع الذي يتأكد من صحة التوقيت المدخل
18.         if (checkTimeValidity(hours, minutes, seconds)) {
19.             // نقوم بعمل object في حالة صحة الأرقام
20.             Time time = new Time(hours, minutes, seconds);
21.             System.out.println("The time is valid: " + time);
22.         } else {
23.             System.out.println("The time entered is invalid.");
24.         }
25.     }

```

File: Time.java

```

1. Package oop_concept;
2. //Time Class
3. public class Time {
4.     /*The Attributes*/
5.     int seconds;
6.     int minutes;
7.     int hours;
8.     /*Parameterized Constructor*/
9.     public Time(int hours, int minutes, int seconds) {
10.         this.hours = hours;
11.         this.minutes = minutes;
12.         this.seconds = seconds;
13.     }
14.
15.     @Override
16.     public String toString(){
17.         return this.hours + ":" + this.minutes + ":" + this.seconds;
18.     }
19. }

```

```

26. static boolean checkTimeValidity(int
hours, int minutes, int seconds) {
27.     // يجب أن تكون الساعة بين 0 و الثالثة و
العشرين
28.     if (hours < 0 || hours > 23) {
29.         return false;
30.     }
31.
32.     // يجب أن تكون الدقائق المدخلة بين 0 و 59
33.     if (minutes < 0 || minutes > 59) {
34.         return false;
35.     }
36.
37.     // يجب أن يكون عدد الثواني المدخلة بين 0 و 59
38.     if (seconds < 0 || seconds > 59) {
39.         return false;
40.     }
41.
42.     // إذا لم يتم خرق أي من الحالات أعلاه يكون
الرقم صحيحاً و جاهزاً لنقوم بعمل أوبجكت منه
43.     return true;
44. }
45. }

```

ملاحظة (تابع toString):

تابع موجود بكللاس Object الذي ترث منه كل كلاسات Java بشكل تلقائي ، وظيفته القيام بإرجاع string يصف ال object الذي تم استدعاء التابع من خلاله.

و هو بوضعه الطبيعي لا يقوم سوى بإرجاع Class name معبراً عن نوع الكائن متبوعاً ب memory address يعبر عن مكان الكائن بالذاكرة.

لكن بما أنه تابع موجود بكللاس الأب لجميع الكلاسات يمكننا عمل Override له و طباعة المعلومات المفيدة الخاصة بكل object تبعاً لنوعه.

فمثلاً لو لدينا class طالب يحوي اسمه و عمره ورقمه التسلسلي سأجعل هذا التابع يرجع لي هذه البيانات بدلاً من class name ومكان ال object بالذاكرة مما يقدم معلومة مفيدة عن ال object وقابلة للعرض بصورة يمكن استخدامه بها.

ويتم استدعاء هذا التابع تلقائياً عند طباعة ال object باستخدام System.out.println() أو أي نوع من توابع ال print في جافا.

التمرين الثالث:

عرف classes 3 ، هم:

- Engine Class : يحوي: Stop and Start Methods
- Accelerator Class : يحوي: Speed attribute with set&get Methods
- Car Class : يحوي:

Attributes: color – doorsNumber – speed

Methods: Set&Get for each attribute except the speed

setStart:

نعمل فيها object من نوع (محرك) engine و يستخدم start method الخاصة بالمحرك.

carSpeed:

نعمل فيها Object من نوع (دواسة الوقود) accelerator نستخدم ضمنه set&get
ضمنه نعطي الدواسة سرعة ثم نأخذها و نسندها لسرعة السيارة.

carFetures:

تطبع خصائص السيارة الأساسية من لون و سرعة و عدد أبواب.

وفي Main Class: التابع الرئيسي نعمل ضمنه OBJECT من نوع CAR
نستخدم كافة توابعه .set&get,setStart,carSpeed,carFeatures.

الحل:

File: Car.java

```
1. Package oop_concept;
2. //Car Class
3. public class Car {
4.     //attributes
5.     private int doors;
6.     private String color;
7.     private int speed;
8.
9.     //set&get methods
10.    public int getDoors(){
11.        return doors;
12.    }
13.    public void setDoors(int doors){
14.        this.doors=doors;
15.    }
16.    public String getColor(){
17.        return color;
18.    }
19.    public void setColor(String color){
20.        this.color=color;
21.    }
22.
23.    //start car method
24.    public void setStart() {
25.        // كائن من المحرك
26.        Engine engine = new Engine();
27.        //تشغيل المحرك
28.        engine.start();
29.    }
30.    public void carSpeed() {
31.        //كائن من دواصة الوقود
32.        Accelerator accelerator = new Accelerator();
33.        //نعطي قيمة سرعة السيارة المراد أن نصل لها في هذا التابع
34.        accelerator.setSpeed(100);
35.        //نسند لسرعة السيارة قيمة السرعة التي أعطيناها
36.        //كائن دواصة الوقود
37.        this.speed = accelerator.getSpeed();
38.    }
39.    // car features method
40.    public void carFeatures() {
41.        System.out.println("Car Color= " + color + ",
42.        doors= " + doors + ", speed= " + speed);
43.    }
44. }
```

File: Accelerator.java

```
1. Package oop_concept;
2. //Accelerator Class
3. public class Accelerator {
4.     private int speed;
5.     //إعطاء سرعة للسيارة
6.     public void setSpeed(int speed){
7.         this.speed=speed;
8.     }
9.     //الحصول على سرعة السيارة
10.    public int getSpeed(){
11.        return speed;
12.    }
13. }
```

File: Engine.java

```
1. Package oop_concept;
2. //Engine Class
3. public class Engine {
4.     //تشغيل المحرك
5.     public void start(){
6.         System.out.println("starting the engine");
7.     }
8.     //إطفاء المحرك
9.     public void stop(){
10.        System.out.println("stopping the engine");
11.    }
12. }
```

File: Main

```
1. Package oop_concept;
2. // Main Class
3. public class Main {
4.     public static void main(String[] args) {
5.         //كائن من السيارة
6.         Car kia = new Car();
7.         kia.setColor("Gray");
8.         kia.setDoors(4);
9.         kia.setStart();
10.        kia.carSpeed();
11.        kia.carFeatures();
12.    }
```

التمرين الرابع:

عرف 2 Classes:

▪ Book: يحوي:

Attributes : title and author.

Methods : parameterized constructor.

▪ Library: يحوي:

Attributes : book List.

Methods : parameterized constructor and get method for the list.

وفي Main Class قم بتعريف التابع main و فيه قم بعمل عدد من الكتب ثم أنشئ List منها و قم بعمل مكتبة بتمرير هذه ال List للباقي.

File: Main.java

```

1. Package oop_concept;
2. //(ArrayLists للاستفادة من توابعها (للاستخدام
3. import java.util.*;
4. // Main Class
5. class MainClass {
6. public static void main(String[] args) {
7.     // إنشاء الكتب من خلال تمرير اسمي الكتاب و الكاتب
8.     Book book1 = new Book("EffectiveJ Java", "Joshua Bloch");
9.     Book book2 = new Book("Thinking in Java", "Bruce
Eckel");
10.    Book book3 = new Book("Java: The Complete
Reference", "Herbert Schildt");
11.    // List
12.    // من الكتب
13.    List < Book > books = new ArrayList < Book > ();
14.    books.add(book1);
15.    books.add(book2);
16.    books.add(book3);
17.    // ننشئ مكتبة من خلال تمرير قائمة الكتب السابقة
18.    Library library = new Library(books);
19.    // نستخدم getTotalBooksInLibrary
20.    // للحصول على List الكتب بالمكتبة
21.    List < Book > Books =
library.getTotalBooksInLibrary();
22.
23.    // foreach loop
24.    // لطباعة بيانات كل كتاب على حدة
25.    for (Book bk: Books) {
26.        System.out.println("Title : " + book.title + " and " +
" Author : " + book.author);
27.    }
28. }
29. }
```

File: Book.java

```

1. Package oop_concept;
2. // Book Class
3. import java.util.*;
4. class Book {
5.     //attributes :
6.     public String title;
7.     public String author;
8.     //constructor
9.     Book(String title, String author) {
10.        this.title = title;
11.        this.author = author;
12.    }
13. }
```

File: Library.java

```

1. Package oop_concept;
2. //Library Class
3. import java.util.*;
4. class Library {
5.     //attribute:
6.     private final List < Book > books;
7.     //constructor
8.     Library(List < Book > books) {
9.        this.books = books;
10.    }
11.    // get method لنحصل على قائمة الكتب
12.    public List < Book >
getTotalBooksInLibrary() {
13.        return books;
14.    }
15. }
```

التمرين الخامس:

قم بعمل كلاس Square يحوي:

▪ Attributes: طول الضلع

▪ Methods:

Default constructor : يعطي قيمة افتراضية لطول الضلع.

spaceCalculation : يحسب المساحة لهذا المربع.

perimeterCalculation : يحسب المحيط لهذا المربع.

overriding toString : لطباعة كافة البيانات الخاصة بهذا المربع عند استخدام `system.out.println()`;

وكلاس Main فيه التابع main و فيه قم بعمل كائن من المربع و اطبع بياناته.

الحل:

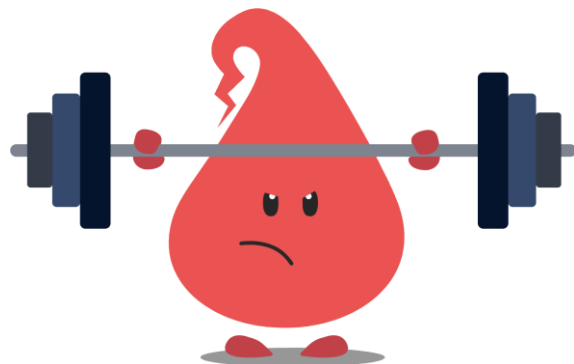
File: Square

```
1. Package oop_concept;
2. //Square Class
3. public class Square {
4.     int sideLength ;
5.     public Square(){
6.         sideLength = 10;
7.     }
8.     public int spaceCalculation(){
9.         return sideLength*sideLength;
10.    }
11.    public int perimeterCalculation(){
12.        return sideLength*4;
13.    }
14.    @Override
15.    public String toString(){
16.        return "The SideLength is : " + sideLength +
17.        "\nthe space is : "+spaceCalculation()+ "\n the
18.        perimeter is : " +perimeterCalculation();
19.    }
20. }
```

File: Main

```
1. Package oop_concept;
2. //Main Class
3. public class Main {
4.     public static void main(String[] args) {
5.         System.out.println(new Square());
6.     }
7. }
```

You are the only person on earth
who can use your ability!.



التمرين السادس:

قم بتعريف class Good (سلعة) يحوي على:

- Attributes:
 - Name اسم السلعة.
 - price سعر السلعة.
 - quantity كمية السلعة.
- Methods:
 - .Parameterized constructor
 - valueCalculation: لحساب قيمة المشتريات (سعر القطعة مضروباً بكميتها).
 - printInformation: لطباعة معلومات القطعة كافةً والقيمة لكل قطعة على حدى

وفي Main Class :

- قم بتعريف التابع main و فيه قم بعمل مصفوفة من السلع تمثل السلع المشتراة ودع المستخدم يدخل بيانات السلع التي يود.
- ثم قم بعمل تابع منفصل لحساب قيمة المواد المشتراة كافةً billCalculation.
- بنهاية التابع main اطبع كافة السلع المشتراة ببياناتها ثم اطبع السعر النهائي.

الحل:

File: Main.java

```
1. Package oop_concept;
2. // تضمين المكاتب للاستفادة من توابعها
3. import java.util.ArrayList;
4. import java.util.List;
5. import java.util.Scanner;
6. //Main Class
7. public class Main {
8.     public static void main(String[] args) {
9.         //scanner object
10.        Scanner scanner = new Scanner(System.in);
11.        // arraylist of goods
12.        List<Good> goods = new ArrayList<>();
13.        // إدخال لعدد السلع التي يود المستخدم شراءها
14.        System.out.println("Enter the number of goods
you want to buy");
15.        int numberOfGoods = scanner.nextInt();
16.        scanner.nextLine();
17.
```

File: Good.java

```
1. Package oop_concept;
2. // Good Class
3. public class Good {
4.     public double price;
5.     public String name;
6.     public int quantity;
7.     //parameterized Constructor
8.     public Good(String name,double
price,int quantity){
9.         this.name =name;
10.        this.price=price;
11.        this.quantity=quantity;
12.    }
13.    //value Calculation method
14.    public double valueCalculation(){
15.        return quantity*price;
16.    }
```

```

18. // إدخال السلع سلعةً سلعةً
19. for(int i =0 ; i < numberOfGoods ;i++)
20. {
21. // constructor ال scanner مباشرةً في ال
    constructor يقوم المستخدم بإدخال القيم أثناء استدعاء ال
    قيمةً قيمةً و يتم تمريرها في المكان المطلوب مباشرةً (عند إدخال
    كل قيمة نضغط Enter حتى يأخذها)
22. System.out.println("enter the name then the
    price for 1 piece then enter the quantity of the good
    number " + (i+1));
23. goods.add(new
    Good(scanner.nextLine(),scanner.nextDouble(),scanner.
    nextInt()));
24. scanner.nextLine();
25. }
26. // نطبع بيانات كل سلعة على حدة
27. for (Good good :goods)
28. good.printInformation();
29. System.out.println();
30. // نطبع سعر الفاتورة النهائي
31. System.out.println("the final value in the bill is
    " + billCalculation(goods));
32.
33. }
34. //billCalculation method
35. public static double billCalculation(List<Good>
    goods){
36. // متغير نخزن به مجموع الأسعار لنحصل على القيمة
    النهائية
37. double theBill=0;
38. // حلقة للمرور سلعة سلعة على جميع السلع بال
    arraylist
39. for(int i= 0 ; i<goods.size();i++)
40. {
41. theBill+= goods.get(i).valueCalculation();
42. }
43. // إرجاع قيمة السعر النهائي
44. return theBill;
45. }
46. }

```

```

17. //printing information method
18. public void printInformation(){
19. System.out.println("Good : "+name
    + "\n its price is : "+price +"\n with
    quantity of : "+ quantity + "\n final value is
    : "+ valueCalculation() );
20. }
21. }

```

التمرين السابع:

- قم بعمل Classes :
Subject and Semester
كل فصل دراسي (Semester) يحوي مجموعة من المواد (subjects) و رقم.
قم بتعريف تابع ضمنه يقوم بطباعة البيانات الخاصة بالفصل.

- إضافة أريسيزية: قم بكتابة الكود بطريقتين:
الأولى طبق فيها مفهوم ال Aggregation
الثانية طبق فيها مفهوم ال Composition

ملاحظة أريسيزية:

الهدف من الطلبين أن يرى المتلقي أن المفهومين السابقين ضمن الطلبين هما مفاهيم برمجية يمكن توظيفها على حسب غاية المبرمج و ليست قواعد ملزمة بالضرورة.

كلا العلاقتين يمثلان نوع العلاقة has-a ، أي أنها علاقة تعبر عن احتواء كائن لكائن آخر بحيث يصبح الكائن الآخر أحد مكونات الكائن الرئيسي.

و يكمن الفرق في أن:

Composition : يفيد التركيب القوي بين الكائنات و هذا يعني أن انتهاء حياة الكائن الرئيسي الذي يحوي الكائنات الفرعية يؤدي لنهاية هذه الكائنات الفرعية، بعبارة أخرى أي أنها لن توجد في أي مكان ضمن البرنامج سوى ضمن الكائن الرئيسي الذي يحويها.

Aggregation : يفيد التركيب الضعيف بين الكائنات و هذا يعني أن انتهاء حياة الكائن الرئيسي الذي يحوي الكائنات الفرعية لن يؤدي لنهاية حياة الكائنات الفرعية و ستبقى في مكان ما في البرنامج قابلة للاستخدام.

الحل بالطريقة الأولى (Aggregation):

File: Subject.java

```

1. Package oop_concept;
2. //Subject Class
3. public class Subject {
4.     //attributes
5.     String name ;
6.
7.     //constructor
8.     Subject (String name){
9.         this.name = name;
10.    }
11.    //toString method
12.    // مباشرة حاصلين من subject لنقوم بطباعة
    خلالها على اسم المادة
13.    @Override
14.    public String toString (){
15.        return this.name ;
16.    }
17. }
```



File: Main.java

```

1. Package oop_concept;
2. import java.util.ArrayList;
3. import java.util.Scanner;
4. // Main Class
5. public class Main {
6.     public static void main(String[] args) {
7.         //scanner object
8.         Scanner scan = new Scanner(System.in);
9.         //semester number
10.        System.out.println("enter the semester
");
11.        int semesterNumber = scan.nextInt();
12.        //عدد المواد
13.        System.out.println("enter the number of
subjects you want to add");
14.        int subjectsNumber= scan.nextInt();
15.        scan.nextLine();
16.        //arraylist
17.        // المواد التي نريد تمريرها لأوبجكت الفصل
الدراسي
18.        ArrayList<Subject> subjects = new
ArrayList<>();
19.        //المستخدم يدخل المواد
20.        System.out.println("enter the names of
the subjects");
21.        for(int i = 0; i < subjectsNumber ; i++)
22.        {
23.            // توضيح : نعمل كائن من المادة الدراسية
في بانيه يتطلب String فنقوم بعمل كائن من String
ونمرر لبانيه scan.nextLine لجعل المستخدم يرسل له
القيمة، فبالنتيجة سيكون لدينا مادة جديدة
24.            Subject subject = new Subject(new
String(scan.nextLine()));
25.            // نضيف لل arraylist
26.            subjects.add(subject);
27.        }
28.        // نعمل كائن من الفصل الدراسي ممررين له
رقم الفصل و مصفوفة المواد
29.        Semester semester = new
Semester(semesterNumber,subjects);
30.        // نطبع معلومات الفصل
31.        semester.printMeth();
32.    }
33. }

```

File: Semester.java

```

1. Package oop_concept;
2. import java.util.ArrayList;
3. //Semester Class
4. public class Semester {
5.     //attributes
6.     int number;
7.     ArrayList<Subject> subjects;
8.
9.     //parameterized constructor
10.    Semester (int number,ArrayList<Subject>
subjects){
11.        this.subjects = subjects;
12.        this.number = number;
13.    }
14.    //print method
15.    public void printMeth(){
16.        System.out.println("the semester number
is : "+number + "\n" + "it has the following
subjects :");
17.        for (Subject subject : subjects)
18.            System.out.println(subject) ;
19.        // override اسم المادة لأننا قمنا بعمل
toString
20.    }
21. }

```

ملاحظة أريسية (شرح استخدام (nextLine():

عند استخدام كائن Scanner يقوم بتخزين ما يدخله المستخدم مؤقتاً في شيء يسمى buffer، وهو عبارة عن مصفوفة من البايتات (array of bytes)، ويقوم عندها ال scanner باستهلاك ما يحتويه ال buffer بشكل تدريجي أو دفعة واحدة حسب كم البيانات المدخلة.

عند استخدام nextInt()، nextBoolean()، nextDouble() لقراءة رقم صحيح أو قيمة منطقية boolean أو عدد عشري فإن Scanner يقرأ القيمة المطلوبة فقط ويتجاهل باقي الرموز، مثل المسافات أو محرف newLine الناتج من ضغط Enter، و محرف newline يبقى في ال buffer بعد قراءة القيمة.

بالتالي، عند استخدام nextLine() بعد أي من هذه التوابع، سيقوم nextLine() بقراءة هذا المحرف المتبقي، مما قد يتسبب في أن تكون القيمة المقروءة فارغة، حيث يتم اعتبار محرف newline نصاً قد أدخله المستخدم مما يمنع المستخدم من إدخال النص الخاص به.

ولحل هذا الإشكال نستدعي تابع nextLine() بدون إسناد قيمته لأي متغير. وهذا يكون في الحالة التي يتم فيها استخدام للتوابع التي ذكرنا أنها تترك محرف newline و ثم نريد أن ندخل نصاً ما باستخدام nextLine()، عند هذا نضع بين التابعين Scanner.nextLine()، بهذا سيتم استهلاك المحرف و يقوم المستخدم بإدخال النص التالي بشكل طبيعي.

مثال من الكود السابق:

```
int subjectsNumber= scan.nextInt();
    scan.nextLine();
```

يتلو هذا إدخال ل strings باستخدام nextline أثناء عمل subjects بالحلقة فوضعنا nextLine لتستهلك المحرف المتبقي.

الحل بالطريقة الثانية (Composition):

File: Subject.java

```
1. Package oop_concept;
2. //Subject Class
3. public class Subject {
4.     //attributes
5.     String name ;
6.
7.     //constructor
8.     Subject (String name){
9.         this.name = name;
10.    }
11.    //toString method
12.    نقوم بطباعة subject مباشرة حاصلين من //
    خلالها على اسم المادة
13.    @Override
14.    public String toString (){
15.        return this.name ;
16.    }
17. }
```

File: Semester.java

```
1. Package oop_concept;
2. //Semester Class
3. import java.util.ArrayList;
4. public class Semester {
5.     //attributes
6.     int number;
7.     ArrayList<Subject> subjects ;
8.
9.     //parameterized constructor
10.    public Semester (int number){
11.        subjects = new ArrayList<>();
12.        this.number = number;
13.    }
14. }
```

File: Main.java

```

1. Package oop_concept;
2. import java.util.ArrayList;
3. import java.util.Scanner;
4.
5. // Main Class
6. public class Main {
7.     public static void main(String[] args) {
8.         //scanner object
9.         Scanner scan = new
Scanner(System.in);
10.        //semester number
11.        System.out.println("enter the
semester");
12.        int semesterNumber = scan.nextInt();
13.        // نعمل كائن من الفصل الدراسي ممررين
له رقم الفصل، سيؤدي ذلك لإنشاء مصفوفة من
المواد داخل اوبجكت الفصل
14.        Semester semester = new
Semester(semesterNumber);
15.        //عدد المواد
16.        System.out.println("enter the number
of subjects you want to add");
17.        int subjectsNumber= scan.nextInt();
18.        scan.nextLine();
19.        //ترك المستخدم يدخل المواد
20.        System.out.println("enter the names
of the subjects");
21.        for(int i = 0; i < subjectsNumber ; i++)
22.        {
23.            //addSubjects التابع Sting نرسل
ليقوم بإضافة مادة جديدة للقائمة الخاصة بالمواد
ضمن كائن الفصل الدراسي
24.            semester.addSubjects(new
String(scan.nextLine()));
25.        }
26.
27.        // نطبع معلومات الفصل
28.        semester.printMeth();
29.    }
30. }

```

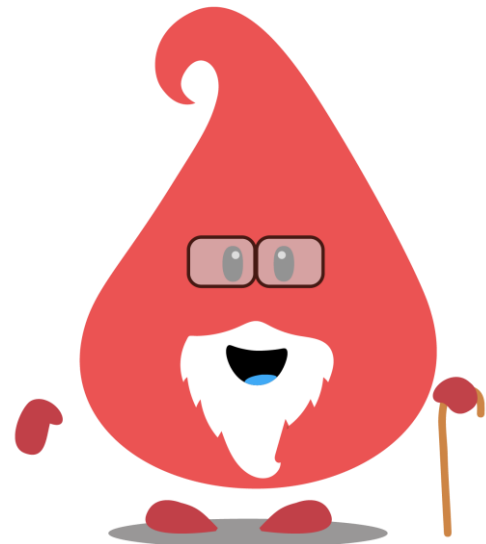
```

15. //add Method
16. public void addSubjects(String subjectName){
17.     //String الماده لمصفوفة المواد بناء على ال
الذي مرره المستخدم
18.     subjects.add(new Subject(subjectName));
19. }
20. //print method
21. public void printMeth(){
22.     System.out.println("the semester number is :
"+number + "\n" + "it has the following subjects : " );
23.     for (Subject subject : subjects)
24.         System.out.println(subject) ;
25.     //override سيطلع اسم المادة لأننا قمنا بعمل
toString ل
26. }
27. }

```

هي المعالي إذا ما رُمتَ تُدركها

فاعبر إليها على جسر من التعب



The End